Big o Notation = describe the performance or complexity of an algorithm

---

Algorithm Sort and Search

Insert

```
n=array.length-1
k=location
item=integer to be inserted


j=n;
while(j>=k);
{
array[j+1]=array[j];
j--;
}

n=n+1;
array[k]=item
```

---

delete

```
n=array.length-1
k=location

j=k;

while(j<N);
{
array[j]=array[j+1];
j++;
}

n=n-1;
```

---

bubble sort

```
n=array.length
i=counter 1
j=counter 2


for(a=0;a<n-1;a++)
{
for(b=0;b<n-a-1;b++)
{
If(array[j]>array[j+1]
{
temp=array[b];
array[b]=array[b+1];
array[b+1]=temp;
}
}
}
```

Balloon sort

```
n=array.length
i=counter 1
j=counter 2


for(a=0;a<n;a++)
{
for(b=0;b<n-1;b++)
{
If(array[j]>array[j+1]
{

temp=array[b];
array[b]=array[b+1];
array[b+1]=temp;
}
}
}
```

Insertion sort

```
n=array.length;
i=counter 1
j=counter 2

for(int i=1;i<n;i++)
{
int temp=array[i];
int j;
for(j=i-1;(j>=0)&&(temp<array[j]);j--)
{
array[j+1]=array[j];
}
array[j+1]=temp;
}
```

Insertion Sort 2

```
for(int x=1; x < arr.length; x++)
  for(int y=x; y > 0 && arr[y-1] > arr[y]; y--){
     int t = arr[y];
     arr[y] = arr[y-1];
     arr[y-1] = t;
```

Sequential Search

```
search=number to be search
For(int i=0;i<array.length;i++)
{
        If(array[i]==search)
                {
                System.out.println("number found at
location "+i);
                Sum+=1;
                }
}

If(sum==0) System.out.print("not found in the array");
Else  System.out.println("total item found "+sum);
```

Binary Search

```
Searh=number to be search;

Low=0;
High=n-1;
Middle=(low+high)/2;

While(low<=high)
{
If(array[middle]<search)
        { low=middle+1;}
Else if(array[middle]>search)
        { high=middle-1;}}
Else System.out.println("found at location"+middle);
```

ALGO

```
If(low>high)
System.out.println("number not found in the array");
```

Growth Rate	Name
1	Constant
log(n)	Logarithmic
n	Linear
n*log(n)	Linearithmic
n^2	Quadratic
n^3	Cubic
2^n	Exponential

Growth Rate	Name	Code e.g.	description
1	Constant	a+=1;	statement (one line of code)

log(n)	Logarithmic	`while(n>1){`
		`  n=n/2;`
		`}`
			Divide in half (binary search)

n	Linear	`for(c=0; c<n; c++){`
		`  a+=1;`
		`}`
			Loop

n*log(n)	Linearithmic	Mergesort, Quicksort, ...	Effective sorting algorithms

n^2	Quadratic	`for(c=0; c<n; c++){`
		`  for(i=0; i<n; i++){`
		`  a+=1;`
		`  }`
		`}`
			Double loop

n^3	Cubic	`for(c=0; c<n; c++){`
		`  for(i=0; i<n; i++){`
		`  for(x=0; x<n; x++){`
		`  a+=1;`
		`  }`
		`  }`
		`}`
			Triple loop

2^n	Exponential	Trying to braeak a password generating all possible combinations	Exhaustive search

Algorithm	best	average	worst	space complexity
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge sort	O(n log(n))	O(n log(n))	O(n log(n))	O(n)